

COMP1511: Programming Fundamentals

L. Cheung

May 4, 2026

Contents

1	Lecture 1	3
1.1	What is COMP1511	3
1.2	Introduction to C	3
2	Lecture 2	4
2.1	How does a computer remember things?	4
2.2	Variables	4
2.3	Input	5
2.4	Constants	5
2.5	Maths	5
3	Lecture 3	7
3.1	If Statements	7
3.2	Operators	7
3.2.1	Relational Operators	7
3.2.2	Logical Operators	8
3.2.3	Example Problem	8
3.3	Looping	9
3.3.1	Example Problem (continued)	9
4	Lecture 4	11
4.1	Examples of if statements and loops	11
4.2	Structures	12
4.3	Enumerations	13
5	Lecture 5	15
5.1	Functions	15
5.2	Style	17
5.2.1	Shorthands	17
6	Lecture 6	19
6.1	Arrays	19
7	Lecture 7	20
7.1	Strings	20
8	Lecture 8	22
8.1	Arrays (continued)	22
9	Lecture 9	23

10 Lecture 10	24
10.1 Command Line Arguments	24
10.2 Pointers	25
10.3 Memory	25
11 Lecture 11	26
11.1 Multi-File Projects	26
11.2 Revisiting Memory	26
11.3 Linked Lists	26
12 Lecture 12	28
12.1 Linked Lists (continued)	28
13 Lecture 13	28
13.1 Inserting into a linked list	28
13.2 Deleting from a linked list	29

1 Lecture 1

1.1 What is COMP1511

- Introduction to programming
- Learning how to write precise instructions to operate computers
- Assumption of no programming knowledge

1.2 Introduction to C

```
#include <stdio.h>

int main(void) {
    printf("Hello world!\n");
    return 0;
}

// Output: Hello world!
```

2 Lecture 2

2.1 How does a computer remember things?

- Computer memory is a big pile of on-off switches, called bits (a choice between a 1 or a 0)
- These bits are usually bunched into sets of 8, a byte
- When executing code, the CPU processes the instructions and performs basic arithmetic, but the RAM keeps track of all the data needed in those instructions and operations

2.2 Variables

- A variable is a certain allocation of bits that can be used to store information
 - `int` → integer, a whole number
 - * A whole number, no fractions or decimals
 - * Most commonly uses 32 bits (4 bytes)
 - * Exactly 2^{32} different values
 - * Finite range
 - `char` → a single character
 - * The character holds an ASCII value, allowing characters to be read as integers
 - * Lowercase letters are only 32 numbers away from their uppercase variant (flipping one bit)
 - * Enclosed using single apostrophes
 - `double` → floating point number
 - * A double-sized floating point number (64 bits, hence double the size of integers)
 - * Floating point means the point can be anywhere in the number
- Names are a description of what the variable is
- C is case sensitive, "ansWer" ≠ "answer"
- C also reserves some words (eg. "return", "int", "double")
- Variables are printed using a format specifier (eg. `%d` formats the variable in base 10, `%lf` formats it in a double, `%c` formats it as a character)
 - When using `%c`, a space can be used (" `%c`") to "swallow" leading whitespace
- When printing variables, they appear in the order of the specifiers given
 - eg. `printf("Age: &d\n Name: %d\n", name, age)`

```
#include <stdio.h>

int main(void) {
    double grade = 99.9;
    int age = 18;           // dcc throws an error for unused variables
    char first_initial = 'H';
    grade = 97.5;
    age = 67;
    printf("%d\n", age);    // "%d" formats the variable as a decimal
}

// Output: 67
```

2.3 Input

- scanf can be used to get an input

```
#include <stdio.h>

int main(void) {
    int age;
    printf("Enter your age: ")
    scanf("%d", &age);
    printf("Your age is %d!\n", age);
    return 0;
}

// Output: Entered number
```

- The & symbol tells scanf the address of the variable in memory and where to place the given value
- Inserting a space before the specifier in scanf tells it to ignore all preceding whitespace

2.4 Constants

- Constants are usually defined at the start of the script using `#define CONST VALUE`

2.5 Maths

- Very familiar functions
 - Adding +
 - Subtracting -
 - Multiplication *
 - Division /

```
#include <stdio.h>

int main(void) {
    int age = 12;
    age = age + 15 * 3;
    printf("%d\n", age);
    return 0;
}

// Output: 57
```

- BODMAS applies to maths in C
- Math can be done to characters since they are just integers
- Adding two large integers may roll over the maximum value and produce a very small or negative number (gcc will throw warning if this occurs)
- There is no infinite precision when encoding a number (eg, a third cannot be represented in binary)
- C will maintain variable types when doing arithmetic
- Integers will drop whatever fraction exists, ie. rounding down

- `%` is called the modulus and will provide the remainder from the division between two integers, eg.
 $5 \% 3 = 2$ since $\frac{5}{3} = 1 \text{ rem } 2$

```
#include <stdio.h>

int main(void) {
    int number = 15;
    int new_number = number % 4;
    printf("%d\n", new_number);
    return 0;
}

// Output: 3
```

3 Lecture 3

3.1 If Statements

- Using if statements, a program can branch between sets of instructions depending on a condition
- Can be used where a decision problem is a question with a yes/no answer

```
if (condition) {  
    do something;  
    do something else;  
}
```

Example.

```
#include <stdio.h>  
  
int main(void) {  
    int number;  
    printf("what is your favourite number: ");  
    scanf("%d", &number);  
    if (number == 15) {  
        printf("That's Henry's birthday!\n")  
    }  
    return 0;  
}
```

- If statements can be chained using `else if` statements

Example.

```
#include <stdio.h>  
if (condition_one) {  
    do something  
} else if (condition_two) {  
    do different_something  
} else {  
    do another_different_something  
}
```

3.2 Operators

3.2.1 Relational Operators

- Relational operators work with pairs of numbers:
 - < less than
 - > greater than
 - <= less than or equal to
 - >= greater than or equal to
 - == equals
 - != not equal to
- All these result in 0 if false and 1 if true

3.2.2 Logical Operators

- Between two expressions:
 - `&&` AND: if both expressions are true then the condition is true
 - `||` OR: if an of the two expressions are true the the condition is true
- In front of an expression:
 - `!` NOT: reverse the expression

Important!!!

Any value that is NOT 0 is TRUE

Example.

```
#include <stdio.h>

int main(void) {
    int order = 66;
    if (order > 60 && order % 2 == 1) {
        printf("The Jedi are safe\n")
    }
    return 0;
}
```

- Brackets can be used to group logic statements to apply an order of operations
 - Eg. `if ((condition_one && condition_two) || condition_three)`

3.2.3 Example Problem

We have decided to run a competition to see how many free energy drinks were given out at O- Week. Students that guess the right number of free energy drinks win! You get told whether your guess was less than, more than or the winning guess :) Extend the problem - if you are within 5 of the correct number, you win the guessing game.

```
#include <stdio.h>

int main(void) {
    int guess;
    printf("Guess how many energy drinks were given out at O-Week: ");
    scanf("%d", &guess);
    int answer = 94;
    if (guess == answer) {
        printf("Congratulations!\n");
    } else if (answer - guess <= 5 && answer - guess >= -5) {
        printf("So close, but no cigar\n");
    } else if (guess < answer) {
        printf("Too small!\n");
    } else {
        printf("That guess is too big!\n");
    }
}
```


3.3 Looping

- C is executed line by line starting from the main function after any
- If statements allow different sections of code to be run, however while loops allow us to repeat sections of code
- `while()` loops can be commonly controlled in three ways:
 - Count loops
 - * Repetition a set number of times

```
#include <stdio.h>

int main(void) {
    int i = 0;
    while (i < 3) {
        printf("Yippee!\n");
        i++;           // Increments int i by 1
    }
}
```

- Sentinel loops
 - * Repetition until a conditional is met
 - * The variable that defines whether or not the loop runs is called the sentinel
 - * The "termination condition" can be checked in the while expression

```
#include <stdio.h>

int main(void) {
    int is_correct = 0;
    while (is_correct == 0) {
        do something
    }
    return 0;
}
```

- Conditional loops
 - * Can also use a condition to decide to exit a loop at any time

3.3.1 Example Problem (continued)

```
#include <stdio.h>

int main(void) {
    int guess;
    printf("Guess how many energy drinks were given out at O-Week: ");
    scanf("%d", &guess);
    int answer = 94;
    int is_correct = 0;
    while (is_correct == 0) {
        if (guess == answer) {
            printf("Congratulations!\n");
        }
    }
}
```

```

        is_correct = 1;
    } else if (answer - guess <= 5 && answer - guess >= -5) {
        printf("So close, but no cigar\n");
    } else if (guess < answer) {
        printf("Too small!\n");
    } else {
        printf("That guess is too big!\n");
    }
}
}

```

4 Lecture 4

4.1 Examples of if statements and loops

Simple If Statement

```
#include <stdio.h>

int main(void) {
    int x = 42;
    if(x == 42 || x % 2 == 0) {
        printf("That's even, or might be the meaning of life\n");
    } else {
        printf("%d is not he meaning of life\n", x);
    }
    return 0;
}
```

Sentinel While Loop

```
#include <stdio.h>

int main(void) {
    int beverage_number;
    int valid_order = 0;
    while (valid_order == 0) {
        printf("What beverage would you like? 0 for tea, 1 for coffee: ");
        scanf("%d", &beverage_number);
        if (beverage_number == 0) {
            printf("Enjoy your green tea!\n");
            valid_order = 1;
        } else if (beverage_number == 1) {
            printf("Enjoy your coffee!\n");
            valid_order = 1;
        } else {
            printf("That is neither tea nor coffee\n");
        }
    }
    return 0;
}
```

Loop de Loop

```
#include <stdio.h>

int main(void) {
    int i = 0;
    int j = 0;
    while (i < 5) {
        while (j < 5) {
            printf("I am printing!\n");
            j++;
        }
        i++;
    }
    return 0;
}
```

Grid of Numbers

Goal: Print a grid of numbers as such:

```
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

Solution:

```
#include <stdio.h>

int main(void) {
    int i = 0;
    while (i < 5) {
        int j = 0;
        while (j < 5) {
            printf("%d ", j + 1);    // Print the value of j
            j++;
        }
        i++;
        printf("\n");                // Loop this 5 times
    }
    return 0;
}
```

4.2 Structures

- Structures can be used to organise related but different components into one structure
- Useful in defining real world problems
- To create a struct:
 1. Define the struct (outside the main)

2. Declare the struct (inside the main)
3. Initialise the struct (inside the main)

```
#include <stdio.h>

// Define the struct
struct coordinate {
    int x_coordinate;
    int y_coordinate;
};

int main(void) {
    // Declare the struct
    struct coordinate my_base;
    my_base.x_coordinate = -12;           // Initialising the struct by using its values
    my_base.y_coordinate = 65;

    struct coordinate looking;
    looking.x_coordinate = -10;
    looking.y_coordinate = 60;

    if (looking.x_coordinate > -15 && looking.x_coordinate < -10) {
        if (looking.y_coordinate >= 55 && looking.y_coordinate <= 70) {
            printf("You're getting close to my base!\n");
        } else {
            printf("You're y coordinate is off\n")
        }
    }

    printf("My base is located at x=%d, y=%d\n", my_base.x_coordinate, my_base.y_coordinate);
    return 0;
}
```

4.3 Enumerations

- Integer data types with a limited range of values (enumerated constants)
- Used to assign names to integral constants
- The index can be modified using flags

```
#include <stdio.h>

// Declaration, NOT assignment (no equal sign)
enum weekdays {MON, TUE, WED, THU, FRI, SAT, SUN};

int main(void) {
    enum weekdays day;
    day = SAT;
    printf("%d", day);
    return 0;
}
```

```
// Output: 5  
// Explanation: Since SAT occupies the fifth index  
// of the enum (starting at 0) it returns 5
```

Using flags

```
#include <stdio.h>  
  
enum states {SUCCESS, FAILURE = 2, UNKNOWN}  
main(void) {  
    enum states flag = UNKNOWN;  
    printf("%d\n", flag);  
    return 0;  
}  
  
// Output: UNKNOWN = 3  
// Explanation: C will fill in the remaining constants with appropriate flags
```

5 Lecture 5

5.1 Functions

- A function is a way of breaking down code into smaller functional parts
 - Each function performs some sort of operation
 - Each function has inputs and an output (these may be empty)
 - The function can be called from anywhere in the code

```
int add (int number_one, int number_two) {  
    int sum;  
    sum = number_one + number_two;  
    return sum;  
}
```

- In the above code, the **return type** is specified before the name of the function, in this case `int`
- The style guide specifies that the `main()` function must come first in the code, however the functions called come after the main. Hence a "prototype" must be initialised before the main function

dice_roll_functions.c

```
#include <stdio.h>

#define TARGET 11

// Function prototype
// A prototype tells the program that the function exists
void print_intro(void);
int sum_dice(int die_one, int die_two);
int win_check(int sum);

int main(void) {
    int die_one;
    int die_two;
    int sum;
    char playing = 'y';

    print_intro();

    while (playing == 1) {

        printf("Enter the first die roll: ");
        scanf("%d", &die_one);

        printf("Enter the second die roll: ");
        scanf("%d", &die_two);

        sum = sum_dice(die_one, die_two);

        if (win_check(sum)) {
            printf("You win!\n")
        } else {
            printf("You lose!\n")
        }

        printf("Roll again? y/n ");
        scanf(" %c", &playing);
    }
    return 0;
}
```


dice_roll_functions.c (continued)

```
// Function that prints the intro
// INPUT: void
// OUTPUT: void
// NAME: print_intro
void print_intro(void) {
    printf("==== Lucky Dice Game ====\\n");
    printf("If you get the target number you win!\\n");
}

// Function that sums two dice rolls
// INPUT: int, int
// OUTPUT: int
// NAME: sum
int sum_dice(int die_one, int die_two) {
    return die_one + die_two;
}

// Function that sees if the sum is the target number
// INPUT: int
// OUTPUT: int
// NAME: win_check
int win_check(int sum) {
    if (sum == TARGET) {
        return 1;
    } else {
        return 0;
    }
}
```

5.2 Style

- Code should be written with readability in mind
- Neat code makes it easier to read and understand

5.2.1 Shorthands

- Increments by 1

```
count = count + 1;
count++;
```

- Decrements by 1

```
count = count - 1;
count--;
```

- Decrement by 5

```
count -= 5;
```

- Multiply by 5

```
count *= 5
```

- `scanf()` returns **the number of things it has successfully scanned in**

```
if (scanf("%d", &size) != 1)
```

6 Lecture 6

6.1 Arrays

- A collection of variables that are all of the same type
- Indexed data structure, therefore accessing and changing variables are simple
- Index starts at 0

```
// Declaring an array
int sample_array[7];    // An array of ints with 7 objects, indexes 0-6

// Declaring and initialising an array
int sample_array[7] = {1, 2, 3, 4, 5, 6, 7}
```

demo_array.c

```
#include <stdio.h>

#define SIZE 10

// Prototype
int find_max(int array[], int size)

int main(void) {
    // 1. Declare an array of size 10, with type ints
    int array_energy[SIZE] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    for (int i = 0; i < SIZE; i++) {
        printf("Enter a number: ");
        scanf("%d", &array_energy[i])
    }
    printf("The maximum of the array is: %d\n", find_max(array_energy, 10))
    return 0;
}

// Function to find the maximum of the array
// INPUTS: int array, int size
// OUTPUT: int maximum_number
int find_max(int array[], int size) {
    int max = array[0];
    for (int i = 1; i < size; i++) {
        if (array[i] > max) {
            max = array[i];
        }
    }
    return max;
}
```

7 Lecture 7

7.1 Strings

- A string is a collection of characters joined together (an array of chars)
- The array finishes with a null terminating character located at the end of the array
- Not displayed, act as a placeholder
- Can be declared using double quotation marks
- `fgets(array, MAX_LENGTH, stdin)` is a function used to read strings
 - `scanf` shouldn't be used as it does not account for buffer overflows or spaces

Example

```
#include <stdio.h>
// Special string functions
#include <string.h>

#define SIZE 10

int main(void) {

    // Declare an array of chars of size 10
    char name[SIZE];
    printf("Enter a name: ");

    // name - name of the array
    // SIZE - maximum size of the array
    // stdin - standard input (scan in from terminal)
    fgets(name, SIZE, stdin);

    // To clear the newline at the end of the buffer
    for (int i = 0; name[i] != '\0'; i++) {
        if (name[i] == '\n') {
            name[i] = '\0'
        }
    }

    // Function strlen()
    // Length of the string (not including the null operator)
    printf("The length of the string: %lu\n", strlen(name))

    // Function strcpy()
    // Copy from one string to another
    char string2[SIZE] = "Hi";
    strcpy(string2, name);
    printf("Copied text is: %s\n", string2);

    // Function strcat()
    // Concatenate two strings
    // Ensure that there is enough space allocated
    strcat(string2, name);
    printf("string2: %s\n", string2);

    // Function strcmp()
    // Compares strings
    if (strcmp(name, "Sasha") == 0) {
        printf("Match!\n");
    } else {
        printf("No match!\n")
    }

    printf("The string is: %s\n", name)
    return 0;
}
```

8 Lecture 8

8.1 Arrays (continued)

- To continue an `fgetc()` until no input is detected, create a while loop that repeats until `fgetc(...) == NULL`

Example

```
#include <stdio.h>

#define SIZE 10

int main(void) {

    char string[SIZE];
    printf("Type a string here: ");

    while (fgetc(string, SIZE, stdin) != NULL) {
        printf("Echo: %s\n", string);
    }

    printf("You exited with Ctrl + D\n");
    return 0;
}
```

- An array of structs can be created, however each struct must be manipulated as an item of the array
- Eg. `team[i].name` rather than `team_member.name`
- `getchar()` gets one character
- Arrays of arrays
 - type `array_name[num of rows][num of columns]`

9 Lecture 9

nothingburger lecture...

Example

```
#include <stdio.h>

#define ROWS 3
#define COLS 4

int max_number(int array[ROWS][COLS]);

int main(void) {
    int array[ROWS][COLS] = {
        {101, 2, 3, 4},
        {5, 6, 7, 8},
        {7, 8, 9, 10}
    };
    int max_num = max_number(array);
    printf("The highest number is %d\n", max_num);
    return 0;
}

int max_number(int array[ROWS][COLS]) {
    int max_num = array[0][0];
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            if (array[i][j] > max_num) {
                max_num = array[i][j];
            }
        }
    }
    return max_num;
}
```

10 Lecture 10

10.1 Command Line Arguments

- Allow inputs into the program from the very beginning
- `int main(void)` accepts no inputs
- To take inputs, main can be modified eg. `int main(int argc, char *argv[])`
 - `argc` counts how many arguments were inputted
 - `*argv[]` is a pointer to the array of strings (the arguments given)
- To pass `int` types to the program, they need to first be converted from strings by using `atoi()` from the `<stdlib.h>` library

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char*argv[]) {

    int sum = 0;

    if (argc < 3) {
        printf("There are not enough arguments to add up!\n");
        return 1;
    }

    // Start at 1 since argv[0] is the program
    for (int i = 1; i < argc; i++) {
        // atoi() converts from a string to an integer
        printf("argv[%d] = %s\n", i, argv[i]);
        sum = sum + atoi(argv[i]);
    }

    printf("The sum is %d\n", sum);
}
```

Example

```
#include <stdio.h>

int main (int argc, char *argv[]) {
    printf("There are %d command line arguments in this program \n", &argc);

    // argv[0] is always the program name
    printf("The program name is %s (argv[0])\n", argv[0]);

    for (int i = 0; i < argc; i++) {
        printf("Argument %d is %s\n", i, argv[i]);
    }
}
```


10.2 Pointers

- A pointer is another variable that stores a memory address of a variable
- This way, variables can be modified directly at the source
- To declare a pointer, it is specified with an asterisk
 - eg. a pointer to an `int` would be `int *pointer;`

```
int number = 2;
int *number_ptr = &number;

// producing
number = 2
number_ptr = 0xFF40 // Example memory address
```

1. Declare a pointer using `*`, specifying where the pointer points to
2. Initialise a pointer by assigning the address to the variable using `&`
3. Dereference the pointer using `*`, ie. going to the address that the pointer variable is assigned to and finding what is at the address

10.3 Memory

- Stack memory is where features of the program are stored (eg. functions created, variables declared)
- When the program ends, it is removed from the stack
- At compile time, stack memory can be allocated
- Heap memory can be controlled and memory can be allocated and deallocated

11 Lecture 11

11.1 Multi-File Projects

- Big programs are often spread out over multiple files, providing benefits such as
 - Improving readability
 - Separation of code
 - Modules can be written and tested separately
- Header file (.h)
 - No running code
 - Function prototypes for functions that will be implemented in the implementation file
 - defines
- Implementation file (.c)
 - Implement the functions that were defined in the header file
- main.c
 - Call functions that may exist in other modules

11.2 Revisiting Memory

- Heap storage is allocated (and deallocated) by the programmer
- The `malloc()` function is used to allocate memory and `free()` deallocates the memory
 - `malloc` returns a pointer to the address where the memory is allocated
 - `sizeof()` returns the size of a variable

```
int *ptr = malloc(x * sizeof(int));
```

where

- `ptr` is a pointer to the start of the allocated memory,
- `x` is the number of elements,
- and `int` is the type of data defined.

11.3 Linked Lists

- Linked lists are dynamically sized
- Each element is called a node
- The nodes can be added or removed anywhere within the list
- The list does not have to be stored in contiguous memory
- Each node holds some data, and a pointer to the next node
- To find the first node, there is a header that points to it (this header is not a node)
- The last node is a null pointer

Example

```
#include <stdio.h>
#include <stdlib.h>

// node with an int and a pointer
struct node {
    int data;
    struct node *next;
};

void print_list(struct node *head);

int main(void) {

    // Get memory to create a node
    struct node *head = malloc(sizeof(struct node));

    // Initialise data = 1 into the head
    head->data = 1;
    head->next = NULL;

    struct node *second_node = malloc(sizeof(struct node));
    second_node->data = 2;
    second_node->next = head;

    head = second_node;

    print_list(head);

    return 0;
}

void print_list(struct node *head) {
    struct node *current = head;
    while (current != NULL) {
        printf("%d\n", current->data);
        head = current->next;
    }
}
```

12 Lecture 12

12.1 Linked Lists (continued)

Example

```
struct node *create_node(int data, struct node *next) {
    struct node *new_node = malloc(sizeof(struct node));
    new_node->data = data;
    new_node->next = next;

    return new_node;
}

struct node *append_node(int data, struct node *head) {
    struct node *new_node = create_node(data, NULL);
    struct node *current = head;

    while(current != NULL) {
        current = current->next;
    }
}
```

13 Lecture 13

13.1 Inserting into a linked list

- Remember to handle the empty list case

```
struct node *current = head;
if (current == NULL) {
    return NULL;
}
```

- To delete one item, the head of the list must be freed

Example

```
struct node *insert_node(int position, int data, struct node *head) {

    // Create node
    struct node *new_node = malloc(sizeof(node));
    new_node->data = data;

    // Empty list case
    if (head == NULL) {
        return new_node;
    } else {
        struct node *current = head;
        for (
            int counter = 1;
            counter != position && current != NULL;
            counter++
        ) {
            current = current->next;
        }

        new_node->next = current->next;
        current->next = new_node;
    }
}
```

13.2 Deleting from a linked list

Example

```
// Delete a node with a given value
struct *node delete_node(int value, struct node head) {
    struct node *current = head;
    struct node *previous = NULL;

    // Loop until value is found
    while (current->data != value && current != NULL) {
        previous = current;
        current = current->next;
    }

    if (current != NULL) {
        if (previous == NULL) {
            head = head->next;
        } else {
            previous->next = current->next;
        }
        free(current);
    }
}
```

14 Lecture 14

nothing...